# ICT 1306
# OBJECT ORIENTED PROGRAMMING

# 8.ABSTRACTION & ENCAPSULATION

TC Irugalbandara
KHA Hettige

# Abstraction

- Abstraction is the process of extracting common features from specific examples.
- Abstraction is a process of defining the essential concepts while ignoring the inessential details.

- Providing only essential information to the outside world and hiding their background details, i.e., to represent the needed information in program without presenting the details.

- Data abstraction is a programming (and design) technique that relies on the separation of interface and implementation.
  - Ex: Driving a car

    Operating a PC

# Example

**Can Do**
turn on
turn off
change the channel
adjust the volume

**Using**
power button
channel changer
volume control

# Types of Abstraction

- **Data Abstraction**

  Programming languages define constructs to simplify the way information is presented to the programmer.

  Ex: Predefined Data types

- **Functional Abstraction**

  Programming languages have constructs that 'gift wrap' very complex and low level instructions into instructions that are much more readable.

  Ex: Functions/method calls

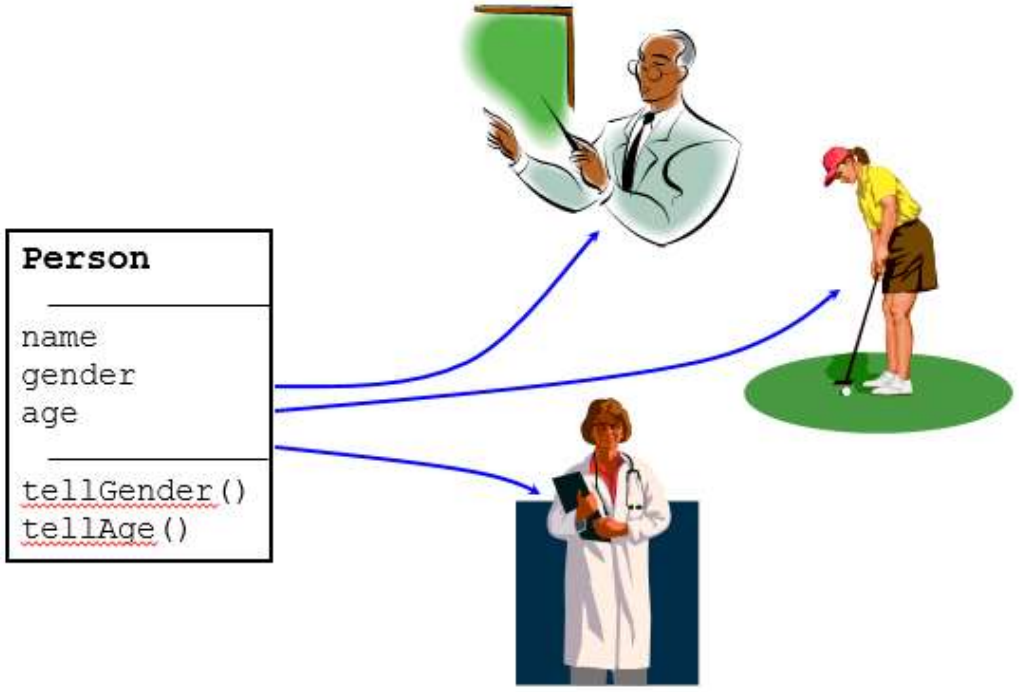
- **Object Abstraction**

  OOP languages take the concept even further and abstract programming constructs as *objects*.

  Ex: Driving a car

# Class as an Abstraction

- A class is an abstraction of its instances. It defines all the attributes and methods that its instances must also have.



- Classes provides great level of **data abstraction**.
- They provide sufficient public methods to the outside world to play with the functionality of the object and to manipulate object data, i.e., state without actually knowing how class has been implemented internally.
- Ex: Sort(), cout
- The underlying implementation of the functionality could change ,as long as the interface stays the same, your function call will still work.

We use **classes** to define our own abstract data types (ADT)

# Example

```
#include <iostream>
using namespace std;
class book
{
        private:
         int bookID;
         int num_of_copies;
         float price;
        public:
         void setbook(int mn, int pn, float c)    {
                 bookID = mn;
                 num_of_copies = pn;
                 price = c;
          }
         void showbook()  {
              cout << "BookID"    << bookID;
              cout << ", Number of Copies "   << num_of_copies;
              cout << ", Unit Price $" << Price ;
           }
};

int main() {
        book book1;
        book1.setbook(1234, 50, 350.50F);
        book1.showbook();
        return 0;
}
```

Out Put:
BookId1234,Number of Copies50,Unit Price $350.50

# Example

```
#include <iostream>
using namespace std;
class book
{
        private:
         int bookID;
         int num_of_copies;
         float price;
        public:
         void setbook(int mn, int pn, float c)    {
                bookID = 1000+mn;
                num_of_copies = pn;
                price = c;
          }
         void showbook()  {
              cout<<"Details of Book :"<<bookID;
              cout << "\nCopies: "   << num_of_copies;
              cout << "\n Price:" << Price ;
            }
};
int main() {
        book book1;
        book1.setbook(1234, 50, 350.50F);
        book1.showbook();
        return 0;
}
```

Out Put:
Details of Book : 1234
Number of Copies: 50
Price: 350.50

# Benefits of Data Abstraction

1. Class internals are protected from inadvertent user-level errors, which might corrupt the state of the object.
2. The class implementation may evolve over time in response to changing requirements or bug reports without requiring change in user-level code.

- Abstraction separates code into interface and implementation.
- Keep interface independent of the implementation so that if you change underlying implementation then interface would remain intact.
- In this case whatever programs are using these interfaces, they would not be impacted .

# Interfaces

- An interface describes the behavior or capabilities of a class without committing to a particular implementation of that class.
- The C++ interfaces are implemented using abstract classes.
- A class is made abstract by declaring at least one of its functions as pure virtual function. A pure virtual function is specified by placing "= 0" in its declaration as follows:

```
class Shape                                /
{
    protected:
       float l;
    public:
       void get_data()
       {
           cin>>l;
       }

       virtual float area() = 0;
};
```

- The purpose of an **abstract class(ABC)** is to provide an appropriate base class from which other classes can inherit.
- Abstract classes cannot be used to instantiate objects and serves only as an **interface**.
- Attempting to instantiate an object of an abstract class causes a compilation error.
- Thus, if a subclass of an ABC needs to be instantiated, it has to implement each of the virtual functions, which means that it supports the interface declared by the ABC.
- Failure to override a pure virtual function in a derived class, then attempting to instantiate objects of that class, is a compilation error.
- Classes that can be used to instantiate objects are called **concrete classes**.

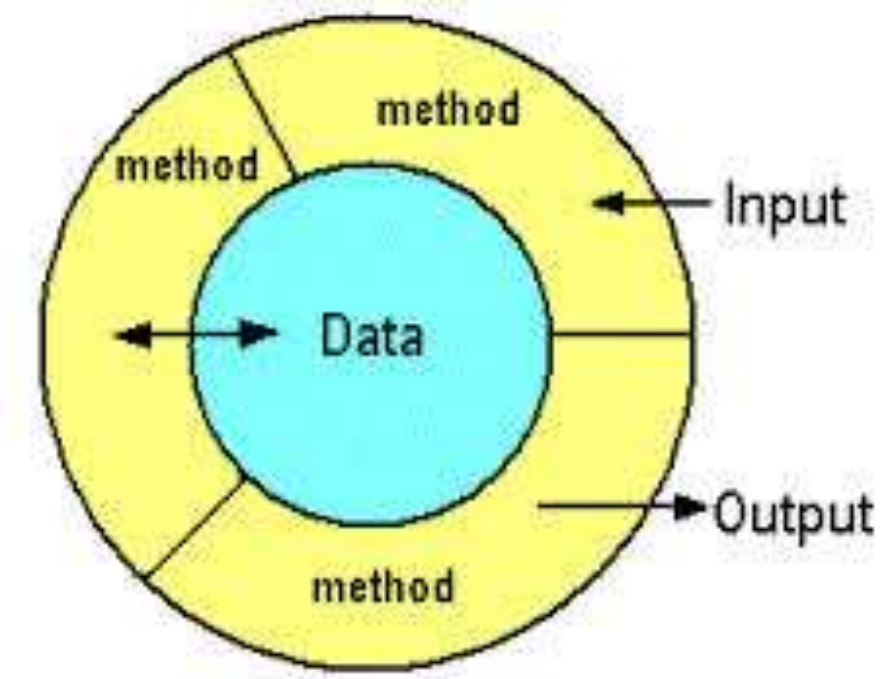
# Encapsulation

- Encapsulation is an Object Oriented Programming concept that binds together the data and functions that manipulate the data, and that keeps both safe from outside interference and misuse.
- Data encapsulation led to the important OOP concept of **data hiding**.

**Data encapsulation** is a mechanism of bundling the data, and the functions that use them

**Data abstraction** is a mechanism of exposing only the interfaces and hiding the implementation details from the user.

- In OOP, classes allow to encapsulate data within the functions.
- Done using access modifiers.
  - Private,Public,Protected
  - By default, all items defined in a class are private.



# Example

```
#include <iostream>

using namespace std;

class book

{

        private:

         int bookID;

         int num_of_copies;

         float price;

        public:

         void setbook(int mn, int pn, float c)    {

                        bookID = mn;
                        num_of_copies = pn;

                        price = c;

          }

         void showbook()  {

                        cout<<"Details of Book :"<<bookID;

                        cout << "\nCopies: "   << num_of_copies;

                        cout << "\n Price:" << Price ;

                  }

};

int main() {

                book book1;

                book1.setbook(1234, 50, 350.50F);

                book1.showbook();

                return 0;

 }
```

- Members of a class must always be declared with the minimum level of visibility.
- Provide setters and getters (also known as accessors/mutators) to allow controlled access to private data.
- Provide other public methods (known as interfaces ) that other objects must adhere to in order to interact with the object.
- Making one class a friend of another reduces encapsulation.
- The ideal is to keep as many of the details of each class hidden from all other classes as possible.

# Setters and Getters

- Setters and Getters allow controlled access to class data
- *Setters* are methods that (only) alter the state of an object
  - Use setters to validate data before changing the object state
- *Getters* are methods that (only) return information about the state of an object
  - Use getters to format data before returning the object's state

```
Private:
    int age;
```

```
Public:
    void setAge(int a){
        // validate
        age=a;
    }

    int getAge(){
        // format
        return age;
    }
```

# Thank You !